

IU Internation University of Applied
Sciences

M.Sc. Computer Science [120 ECTS]
Python Assignment on

“Python's Role in Data Computation Despite Execution Speed
Limitations”

Student Name: Dishan Dalsukhbhai Kheni

Matriculation Number: 102303541

Date: 25th December, 2024

Table of contents

1. Abstraction	03
2. Introduction	04
3. Python's Characteristics and Speed Limitations	05
Interpreted Execution Dynamic typing Global Interpreter Lock (GIL) High-Level Abstraction	
4. Python's Strength in Data Computation Despite Its Slow Nature	08
Extensive Library Ecosystem Integration with Low-Level Languages Developer Productivity and Flexibility Parallelism with External Libraries GPU Acceleration Community and Ecosystem	
5. Python's Strengths to Large Datasets and Dataset Selection	11
Handling Large Datasets Efficiently Dataset Exploration and Visualization	
6. Conclusion	12
7. References	13

1. Abstract

Python is often criticized for its slower execution speed compared to compiled languages like C++ and Rust. Nevertheless, it has emerged as the leading programming language for data computation and machine learning, including tasks like training large language models (LLMs). This paper investigates the factors behind Python's dominance, emphasizing its extensive ecosystem of optimized libraries such as TensorFlow and PyTorch, which mitigate performance limitations while maintaining Python's user-friendly syntax. Additionally, the paper explores Python's unparalleled ease of use, productivity in rapid prototyping, and scalability through tools like GPU acceleration and distributed computing frameworks. Through case studies in LLM training and practical applications, the paper demonstrates how Python's strengths consistently outweigh its inherent drawbacks, securing its position as the preferred language for computationally intensive tasks.

2. Introduction

Python has become a dominant programming language across diverse domains, particularly in data science, machine learning, and artificial intelligence. Its simplicity and vast library ecosystem have made it the top choice among researchers, engineers, and developers. Python's prominence is especially evident in computationally intensive tasks such as the training of large language models (LLMs) like GPT and BERT, which require immense computational power and scalability.

At first glance, Python's widespread adoption in these domains seems paradoxical. As an interpreted language, Python is inherently slower in execution compared to compiled languages like C++ and Rust. Challenges such as the Global Interpreter Lock (GIL) and its single-threaded nature exacerbate its performance bottlenecks. These limitations raise a key question: why has Python maintained its dominance in tasks where raw computational efficiency is critical?

The answer lies in Python's robust ecosystem, which includes highly optimized libraries such as TensorFlow, PyTorch, and NumPy. These libraries integrate seamlessly with faster backends written in C, C++, or CUDA, allowing Python to act as an accessible interface while delegating performance-critical operations to optimized implementations. Additionally, Python's ease of use, versatility, and strong community support enable rapid prototyping, scalability, and efficient collaboration, which are essential in fast-paced research and development environments.

This paper explores the factors contributing to Python's continued dominance despite its performance constraints. It begins with an analysis of Python's inherent limitations and discusses how its ecosystem compensates for these drawbacks. The discussion then shifts to real-world applications, including LLM training, where Python's advantages outweigh its challenges. Finally, the paper assesses Python's future prospects in light of emerging competitors and technological advancements, offering a comprehensive perspective on its role in data computation.

3. Python's Characteristics and Speed Limitations

Python is an interpreted, high-level programming language, valued for its simplicity and versatility. However, its design prioritizes developer productivity and ease of use over raw execution speed, leading to several limitations in computational performance.

1. Interpreted Execution

Python is an interpreted language, which means its code is executed line by line through an interpreter (e.g., CPython). Unlike compiled languages like C++, Python does not convert source code directly to machine code. Instead, it is translated into bytecode, which is then executed by the Python Virtual Machine (PVM).

Impact: This additional interpretation step introduces significant runtime overhead.

Example: Loop Execution

Python	C++
<pre>import time start = time.time() for i in range(10**7): pass end = time.time() print(f"Python Loop Time: {end - start:.2f} seconds") Equivalent loop in C++:</pre>	<pre>#include <iostream> #include <chrono> int main() { auto start = std::chrono::high_resolution_clock::now(); for (int i = 0; i < 10000000; ++i) {} auto end = std::chrono::high_resolution_clock::now(); std::cout << "C++ Loop Time: " << std::chrono::duration<double>(end - start).count() << " seconds\n"; return 0; }</pre>
~1.2 seconds	~0.01 seconds

2. Dynamic Typing

Python uses dynamic typing, meant variable types are determined at runtime. This flexibility allows developers to write concise and versatile code but imposes a heavy computational burden due to constant type checking during execution.

Impact: Each operation must validate operand types dynamically, which slows down execution.

Example: Type Handling

<pre>x = 10 # Initially an integer</pre>
<pre>x = "text" # Dynamically changes to a string</pre>

This dynamic nature requires Python to manage metadata about objects and perform type checks every time an operation is performed.

Comparison: In C++, types are determined at compile time, enabling optimized machine code generation without runtime checks.

3. Global Interpreter Lock (GIL)

The Global Interpreter Lock (GIL) in CPython is a mutex that ensures only one thread executes Python bytecode at a time, even on multi-core processors. This design simplifies memory management but severely limits true parallelism in multi-threaded programs.

Impact: CPU-bound tasks in Python cannot leverage multiple cores effectively, making Python unsuitable for high-performance multi-threaded applications.

Example: Multithreading Limitation

<pre>import time def task(): total = 0 for i in range(10**7): total += i start = time.time() threads = [threading.Thread(target=task) for _ in range(4)] for t in threads: t.start() for t in threads: t.join() end = time.time() print(f"Python Multithreaded Time: {end - start:.2f} seconds")</pre>	<pre>#include <iostream> #include <thread> #include <chrono> void task() { long long total = 0; for (int i = 0; i < 10000000; ++i) { total += i; } } int main() { auto start = std::chrono::high_resolution_clock::now(); std::thread t1(task), t2(task), t3(task), t4(task); t1.join(); t2.join(); t3.join(); t4.join(); auto end = std::chrono::high_resolution_clock::now(); std::cout << "C++ Multithreaded Time: " << std::chrono::duration<double>(end - start).count() << " seconds\n"; return 0; }</pre>
~5 seconds	~1.2 seconds

Reason: Python threads are serialized by the GIL, whereas C++ threads run in parallel.

4. High-Level Abstraction

Python's high-level features, such as automatic memory management, dynamic attribute handling, and introspection, simplify development but introduce computational overhead.

Impact: These abstractions require additional processing at runtime, slowing down performance-critical tasks.

Example: Object-Oriented Overhead

Python	C++
<pre>def __init__(self, value): self.value = value start = time.time() for _ in range(10**6): obj = Example(42) end = time.time() print(f"Python Object Creation Time: {end - start:.2f} seconds")</pre>	<pre>class Example { public: int value; Example(int val) : value(val) {} }; int main() { auto start = std::chrono::high_resolution_clock::now(); for (int i = 0; i < 1000000; ++i) { Example obj(42); } auto end = std::chrono::high_resolution_clock::now(); std::cout << "C++ Object Creation Time: " << std::chrono::duration<double>(end - start).count() << " seconds\n"; return 0; }</pre>
~1.1 seconds	~0.01 seconds

Reason: Python's object creation involves dynamic memory allocation, type handling, and attribute setup, while C++ is optimized for such tasks.

Despite these limitations, Python thrives in computationally intensive tasks due to its ecosystem, which offloads performance-critical operations to faster backend implementations.

4. Python's Strength in Data Computation Despite Its Slow Nature

Python's reputation as a go-to language for data computation may seem paradoxical given its relatively slow performance. However, its design philosophy and ecosystem have equipped it to excel in this domain, making it the preferred choice for data scientists, machine learning engineers, and analysts. Below, we explore Python's strengths for data computation and how it compensates for its inherent slowness.

1. Extensive Library Ecosystem

Python boasts a vast array of optimized libraries for data computation, many of which are written in low-level languages like C or Fortran for performance.

NumPy: Provides fast array manipulations and linear algebra operations, leveraging vectorization and underlying C code.

Pandas: Optimized for data manipulation and analysis, built on top of NumPy.

SciPy: Offers advanced mathematical functions, including numerical integration and optimization.

TensorFlow and PyTorch: Powerhouses for deep learning, utilizing highly optimized backends for computations on CPUs and GPUs.

Proof of Performance: Vectorization with NumPy

Pure Python	NumPy
<pre>import time size = 10**6 a = list(range(size)) b = list(range(size)) start = time.time() result = [a[i] + b[i] for i in range(size)] end = time.time() print(f"Pure Python Time: {end - start:.2f} seconds")</pre>	<pre>import numpy as np start = time.time() a = np.arange(size) b = np.arange(size) result = a + b end = time.time() print(f"NumPy Vectorized Time: {end - start:.2f} seconds")</pre>
~2 seconds	~0.01 seconds

Reason: NumPy's vectorized operations leverage highly optimized C routines, bypassing Python's slow loops.

2. Integration with Low-Level Languages

Python can seamlessly integrate with high-performance languages like C, C++, and Fortran through interfaces such as Cython, SWIG, and F2PY.

Cython: Allows Python code to be compiled into C, dramatically improving execution speed.

Numba: Provides Just-In-Time (JIT) compilation for Python functions, optimizing performance for numerical tasks.

TensorFlow and PyTorch: Use low-level backends (e.g., CUDA) for GPU-accelerated computations.

Proof of Integration: Cython Example

Python	Cython
<pre>def add_numbers(n): result = 0 for i in range(n): result += i return result</pre>	<pre>def add_numbers_cython(int n): cdef int result = 0 cdef int i for i in range(n): result += i return result</pre>

Result: Cython version can be 50–100x faster due to direct compilation into C.

3. Developer Productivity and Flexibility

Python's simple syntax and rich ecosystem allow rapid prototyping and development of data computation workflows.

Ease of Use: Python enables quick exploration of datasets with intuitive libraries like Pandas and Matplotlib.

Interactivity: Tools like Jupyter Notebooks provide an interactive environment for experimentation and visualization.

Example: Pandas for Data Analysis

```
import pandas as pd  
data = {'Name': ['A', 'B', 'C'], 'Age': [20, 21, 21]}  
df = pd.DataFrame(data)  
print(df.describe())
```

Pandas simplifies tasks like filtering, grouping, and aggregating data, which would require verbose code in lower-level languages.

4. Parallelism with External Libraries

While Python's Global Interpreter Lock (GIL) limits multi-threading for CPU-bound tasks, libraries like Dask, Ray, and Joblib enable distributed and parallel computing.

Dask: Extends NumPy and Pandas for scalable, parallel processing.

Ray: Simplifies distributed computing for Python applications.

Proof: Parallel Data Processing with Dask

```
import dask.dataframe as dd
df = dd.read_csv('large_dataset.csv')
result = df.groupby('column_name').sum().compute()
```

Result: Tasks are distributed across multiple cores or machines, bypassing Python's GIL.

5. GPU Acceleration

Python frameworks like TensorFlow, PyTorch, and RAPIDS utilize GPUs to accelerate data computations, making Python capable of handling massive datasets efficiently.

Proof: GPU Computation with PyTorch

```
import torch
a = torch.rand(10000, 10000, device='cuda')
b = torch.rand(10000, 10000, device='cuda')
result = torch.matmul(a, b) # GPU-accelerated matrix multiplication
```

Result: Operations that would take seconds on a CPU can be completed in milliseconds on a GPU.

6. Community and Ecosystem

Python's vibrant community contributes actively to developing tools, frameworks, and tutorials for data computation. This ecosystem ensures constant innovation and widespread adoption.

5. Python's Strengths to Large Datasets and Dataset Selection

In practical assignment, I have to deal with some datasets, where need to load and did little computation to find the best data out of large datasets. And during writing code I observe some python strength as following:

1. Handling Large Datasets Efficiently

When dealing with large datasets, Python's ecosystem offers solutions that mitigate its inherent slowness, enabling efficient processing and exploration:

Efficient Data Loading and Manipulation

- Libraries like Pandas and Dask:
 - Pandas is excellent for in-memory manipulation of medium-sized datasets.
 - Dask extends Pandas to handle datasets that exceed memory limits by processing them in chunks and enabling distributed computation.

2. Dataset Exploration and Visualization

Selecting the best dataset for training or analysis requires thorough exploration and visualization. Python's high-level abstractions simplify these tasks:

- Visualization with Matplotlib and Seaborn

Python offers tools to visualize trends, distributions, and relationships in data. These insights guide decisions on filtering and selecting subsets of data.

6. Conclusion

Python's unique combination of simplicity, versatility, and an expansive ecosystem of optimized libraries has made it the cornerstone of modern data computation, despite its inherent slowness. While its interpreted execution, dynamic typing, and Global Interpreter Lock (GIL) contribute to suboptimal performance in raw computational speed, these limitations are offset by high-performance libraries like NumPy, Pandas, and TensorFlow. These libraries, often implemented in lower-level languages, allow Python to efficiently handle large datasets, perform complex numerical operations, and integrate seamlessly with GPU and distributed computing frameworks.

For large-scale machine learning tasks, Python excels in dataset exploration, preprocessing, feature engineering, and model training. Its ability to leverage vectorized operations, parallel processing, and big data tools ensures scalability and efficiency, making it indispensable for selecting ideal datasets and working with massive data volumes. In balancing developer productivity with computational performance, Python offers a robust and adaptable solution for data-intensive workflows, proving that its strengths far outweigh its inherent limitations.

7. References

Lisandro D. Dalcin, Rodrigo R. Paz, Pablo A. Kler, Alejandro Cosimo, research on parallel computing: <https://www.sciencedirect.com/science/article/abs/pii/S0309170811000777>

Kostiantyn Zhreb: Research on Improving Performance Of Python Code Using Rewriting Rules Technique: https://ceur-ws.org/Vol-2866/ceur_115-125jereb11.pdf

Anthony Peter James Shaw : Paper on Python as a suitable data science:

https://figshare.mq.edu.au/articles/thesis/Towards_evaluating_Python_as_a_suitable_data_science_programming_language_for_modern_computing_architecture/23974764?file=43103554

Lokhande Gaurav, Lavanya Addepalli, Performance of Python Data Visualization Libraries (PDF) [Assessing the Performance of Python Data Visualization Libraries: A Review](#)

Anthony Shaw: Blog on why python is slow:

[Why is Python so slow? | HackerNoon](#)

Alfred "Fred" Christianson, Ways to tackle python performance issues:

<https://www.theserverside.com/tip/How-to-address-Python-performance-problems>

GitHub Repository Link: <https://github.com/Dishank-Kheni/DLMDSPWP01>